

[Project: CONNECT]

게임 기획서

- 학번: 2371172
- 이름: 안태호

데이터 → 화면 변환 로직

[배열 문자 (Monospace)]

[의미]

[게임 내 시각적 상징]

@

플레이어 (Player)



#

벽 (Wall)



.

이동 가능 통로 (Path)



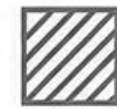
K

열쇠 (Key)



B

장애물 (Block)



*

함정/가시 (Trap)



G

문/출구 (Door/Exit)



O

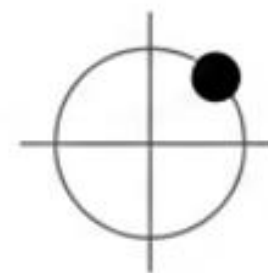
포탈 (Portal)



상태 관리 대시보드

Player State (동적 상태)

int playerX, playerY 실시간 위치 좌표



int playerHP = 5

생명력



int hasKey = 0

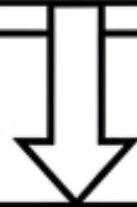
인벤토리 상태
(0: 없음 / 1: 획득)



핵심 로직 I : 초기화 및 렌더링

initMap() (상태 리셋)

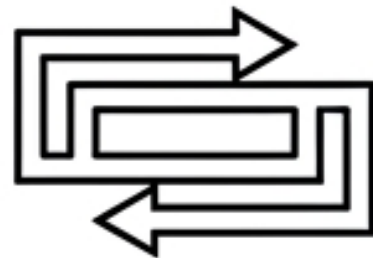
hasKey = 0 ↻
showKeyMsg = 0 ↻
playerHP = 5 ↻



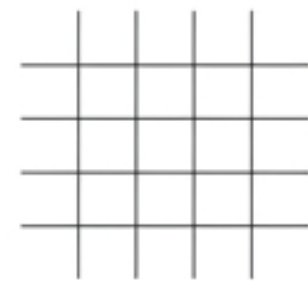
drawMap() (프레임 갱신)

system("cls")

화면 전체 지우기

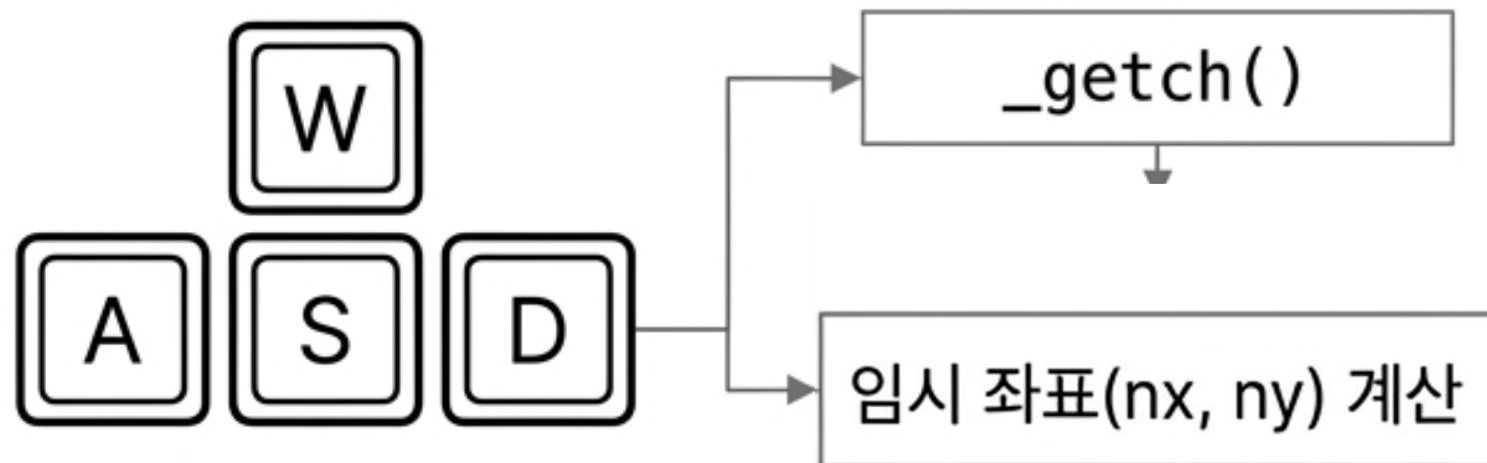


이중 for 루프 실행
(Y축, X축)



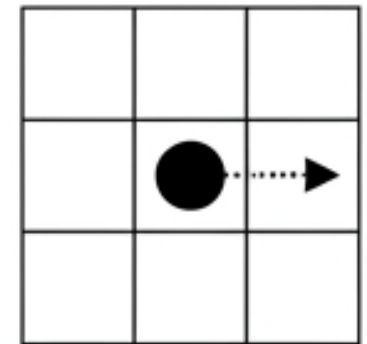
갱신된 map[MAP_H][MAP_W]
배열 콘솔 출력

핵심 로직 II : 이동 및 충돌 판정



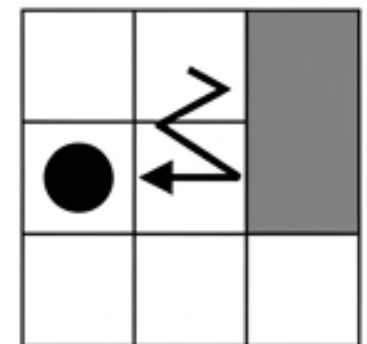
Case 1:
map[ny][nx] == '.'

결과: 이동 가능
-> 실제 좌표 업데이트
(playerX = nx)



Case 2:
map[ny][nx] == '#'

결과: 충돌/이동 불가
-> 업데이트 취소,
제자리 유지



핵심 로직 III : 오브젝트 상호작용

함정 (Trap ***)

플레이어 좌표 == ***

playerHP--



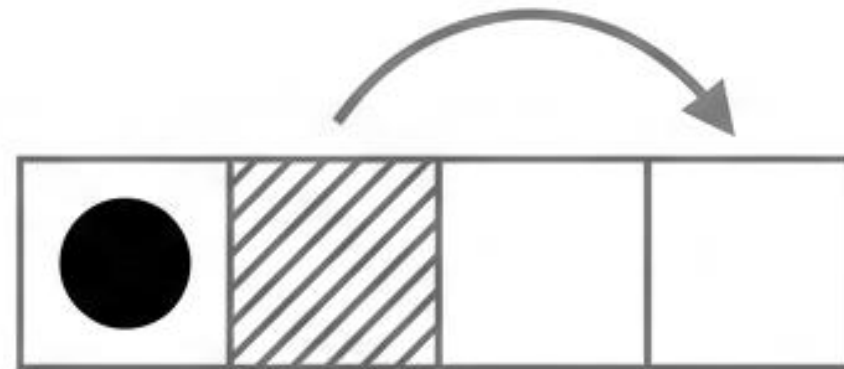
장애물 밀기 (Obstacle B)

플레이어 진행 방향에 B 존재

다음 칸 (' . ') 확인

-> 비어있다면 B 좌표 1칸 이동

-> 플레이어 이동



열쇠와 문 (Key K & Goal G)

K와 충돌

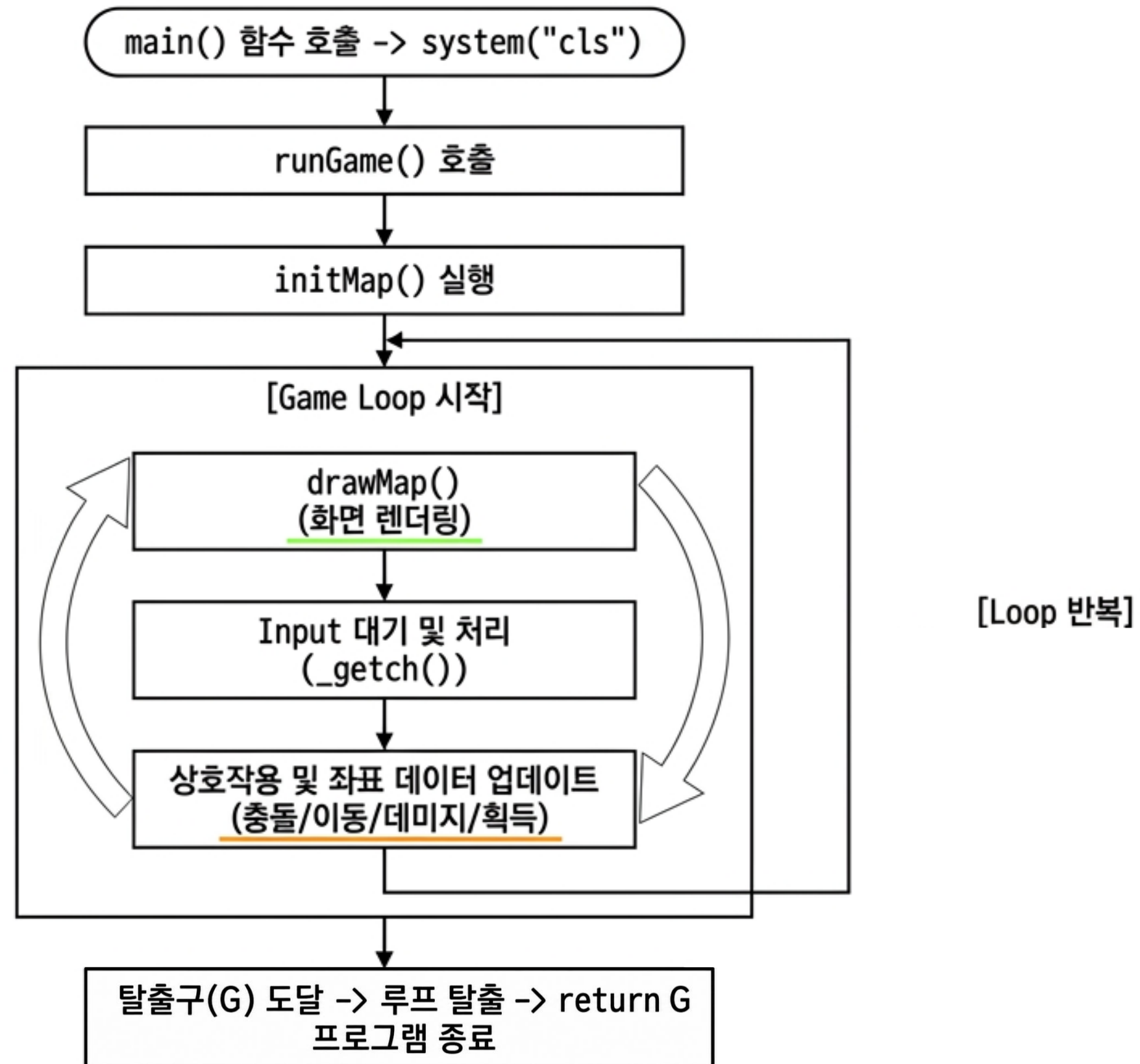
-> hasKey = 1 상태로 변경

G와 충돌 &

hasKey == 1 검사

스테이지 Clear 조건 달성

시스템 실행 흐름도



게임 로직 한눈에 보기

1. 상자를 밀기



플레이어가 상자를 밀어 이동시킬 수 있습니다.

2. 상자와 가시가 만나면 파괴



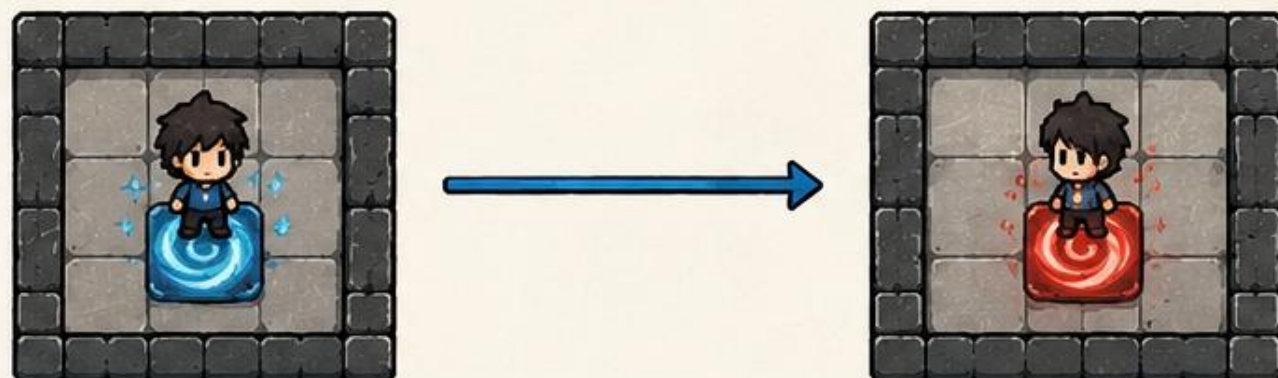
상자가 가시와 만나면 **상자와 가시가 모두 부숩니다.**

3. 키 획득 후 목표 도달 → 클리어



키를 획득한 후 목표 지점(G)에 도달하면 스테이지 **클리어!**

4. 포탈 이동



포탈에 들어가면 **반대편 포탈**로 이동합니다.

타일 설명



플레이어



상자



가시



키



목표



포탈 (입구)



포탈 (출구)



벽

아키텍처 결론 (Conclusion)



Stable Structure

C언어 콘솔 환경의 한계를
system("cls")와 이중 루프
렌더링으로 극복.

Data-Driven Design

3차원 배열(startMap)을 활용하여
코드 수정 없이 데이터만으로
무한한 스테이지 확장 가능.

Logical Scalability

명확히 분리된 상태 변수(State)와
판정 로직을 통해 추가
오브젝트(적, 새로운 함정 등)
확장에 용이한 뼈대 구축.

STAGE 1

HP : ♥ ♥ ♥ ♥ ♥

열쇠 : 없음

```
#####
#.....#
#.....#
#.....Ø...#
#.....#
#.....■ ■ ■.....#
#.....#
#.....***.....#
#.....#
#.....@.....#
#.....#
#.....K.....#
#.....#
#.....G.....#
#.....#
#..O.....#
#.....#
#####
```

이동 : WASD 또는 방향키 | R: 재시작 | ESC: 종료
포탈 : O = 입구 Ø = 출구

|